



Desarrollo App Móviles II

Semana 4: “Actividad: Diagnosticando errores y optimizando el rendimiento”

Índice

1. Introducción y Selección de Flujo	3
2. Ejecución en Modo Depuración y Uso de Logcat.....	3
3. Manejo estructurado de excepciones.....	6
4. Análisis de Rendimiento en Tiempo de Ejecución.....	7
5. Reflexión.....	9

1. Introducción y Selección de Flujo

Para esta actividad, se seleccionó el flujo de "**Procesamiento de Registro de Atención y Pedido de Medicamentos**", localizado en la función **procesarRegistro()** de la clase **RegistroViewModel.kt**.

Justificación de la selección:

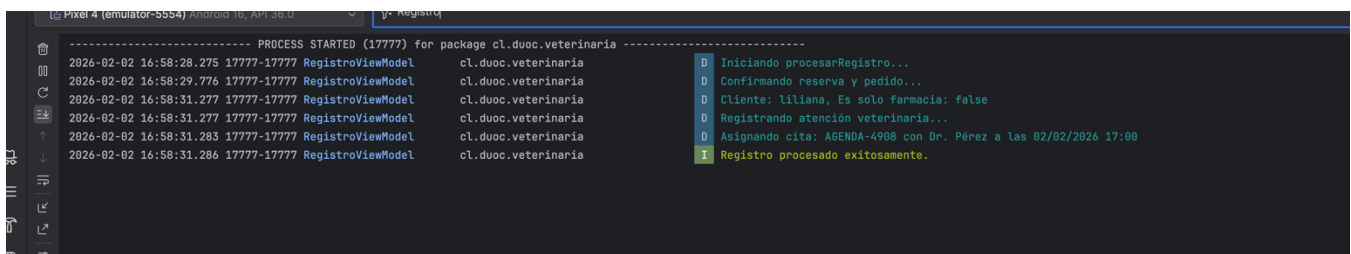
- Complejidad: Es un flujo crítico que combina lógica de negocio, cálculos de costos y persistencia de datos en la base de datos local (Room).
- Uso de Corutinas: Utiliza `viewModelScope.launch` con retardos simulados (`delay`), lo que lo hace ideal para analizar el comportamiento asíncrono.
- Punto de Falla: Al interactuar con recursos externos (repositorio y base de datos), es propenso a excepciones y cuellos de botella.

2. Ejecución en Modo Depuración y Uso de Logcat

Se implementó una trazabilidad completa utilizando la herramienta Logcat. Se definieron etiquetas (TAG = "RegistroViewModel") y mensajes en distintos niveles para monitorear el ciclo de vida del proceso.

Hallazgos durante la depuración:

- Nivel Debug (D): Se confirmaron los hitos del proceso: inicio, identificación del cliente, cálculo de costos y asignación de veterinario.
- Nivel Info (I): Se verificó la finalización exitosa del registro.
- Utilidad: Gracias a estos logs, se pudo determinar que el tiempo de espera entre pasos es de 1.5 segundos, permitiendo verificar que la UI no se bloquea durante este tiempo.



```
----- PROCESS STARTED (17777) for package cl.duoc.veterinaria -----
2026-02-02 16:58:28.275 17777-17777 RegistroViewModel cl.duoc.veterinaria D Iniciando procesarRegistro...
2026-02-02 16:58:29.776 17777-17777 RegistroViewModel cl.duoc.veterinaria D Confirmando reserva y pedido...
2026-02-02 16:58:31.277 17777-17777 RegistroViewModel cl.duoc.veterinaria D Cliente: Liliana, Es solo farmacia: false
2026-02-02 16:58:31.277 17777-17777 RegistroViewModel cl.duoc.veterinaria D Registrando atención veterinaria...
2026-02-02 16:58:31.283 17777-17777 RegistroViewModel cl.duoc.veterinaria D Asignando cita: AGENDA-4988 con Dr. Pérez a las 02/02/2026 17:00
2026-02-02 16:58:31.286 17777-17777 RegistroViewModel cl.duoc.veterinaria I Registro procesado exitosamente.
```

Se validó la traza completa del flujo exitoso, confirmando que todos los hitos del **RegistroViewModel** se cumplieron según lo programado (mensajes en azul y verde). Adicionalmente, se analizó un mensaje de FATAL EXCEPTION (shell-induced crash) que apareció al finalizar la sesión. Se determinó que este no corresponde a un error de código, sino a la interrupción manual del proceso desde Android Studio, confirmando que la aplicación se comporta de manera estable durante su ejecución normal y solo se detiene por acción externa del desarrollador.

```

2026-02-02 18:16:53.544 38375-38375 I/flutter: cl.duoc.veterinaria
2026-02-02 18:16:55.493 38375-38375 RegistroViewModel: cl.duoc.veterinaria
2026-02-02 18:16:57.514 38375-38375 RegistroViewModel: cl.duoc.veterinaria
2026-02-02 18:16:58.495 38375-38375 RegistroViewModel: cl.duoc.veterinaria
2026-02-02 18:16:58.700 38375-38375 RegistroViewModel: cl.duoc.veterinaria
2026-02-02 18:16:58.703 38375-38375 RegistroViewModel: cl.duoc.veterinaria
2026-02-02 18:17:02.498 38375-38375 InsectsController: cl.duoc.veterinaria
2026-02-02 18:17:02.498 38375-38375 InsectsController: cl.duoc.veterinaria
2026-02-02 18:17:03.434 38375-38375 WindowOnBackDispatcher: cl.duoc.veterinaria
2026-02-02 18:17:03.464 38375-38375 HHUI: cl.duoc.veterinaria
2026-02-02 18:17:03.603 38375-38375 HHUI: cl.duoc.veterinaria
2026-02-02 18:19:18.243 38375-38375 AndroidRuntime: cl.duoc.veterinaria
2026-02-02 18:19:18.244 38375-38375 AndroidRuntime: cl.duoc.veterinaria

system_server:886766: onCancelled at PHASE_CLIENT_ON_CONTROLLER_CHANGED
[+] Iniciar proceso de registro...
[+] Confirmando reserva y pedido...
[+] Cliente: Lilianna, Es solo farmacia: false
[+] Registrando atención veterinaria...
[+] Asignando cita: AGENDA-2586 con Dr. Pérez a las 05/02/2026 09:30
[+] Registro procesado exitosamente.
[+] hideLine(): fromMain=false
[+] cl.duoc.veterinaria:918098d: onCancelled at PHASE_CLIENT_ALREADY_HIDDEN
[+] sendCancelIfRunning: isInProgress=false callback=androidx.activity.OnBackPressedDispatcher$Api34Impl$createOnBackPressedAnimationCallback$1@193c05
[+] Image decoding logging dropped!
[+] Image decoding logging dropped!
[+] Shutting down VM
[+] FATAL EXCEPTION: main (App_Semini)
Process: cl.duoc.veterinaria, PID: 38375
android.app.RemoteServiceException$CrashedByYadException: shell-induced crash
    at android.app.ActivityThread.throwRemoteServiceException(ActivityThread.java:2377)
    at android.app.ActivityThread.$$$Nest$throwRemoteServiceException(Unknown Source:0)
    at android.app.ActivityThread$1.handleMessage(ActivityThread.java:2741)
    at android.os.Handler.dispatchMessage(Handler.java:110)
    at android.os.Looper.loopOnce(Looper.java:248)
    at android.os.Looper.loop(Looper.java:338)
    at android.app.ActivityThread.main(ActivityThread.java:9667) [ci internal lines]
    at com.android.internal.os.RuntimeInit$MethodAndArgsCaller.run(RuntimeInit.java:593)
    at com.android.internal.os.ZygoteInit.main(ZygoteInit.java:932)
Sending signal: PID: 38375 SIG: 9

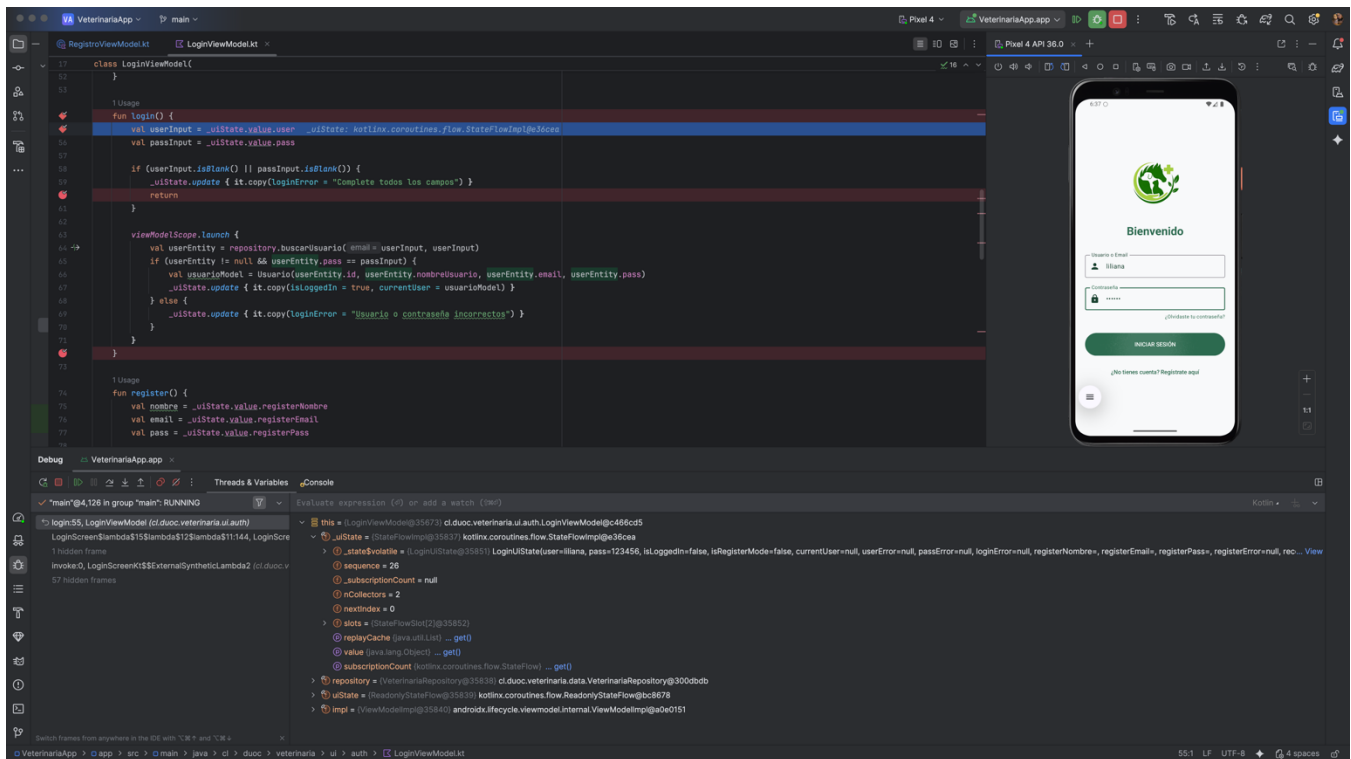
----- PROCESS ENDED (38375) for package cl.duoc.veterinaria -----

```

Inspección de memoria con Breakpoints

Además del monitoreo pasivo con Logcat, se utilizó el Android Debugger para realizar una inspección activa de la aplicación. Se estableció un punto de interrupción (breakpoint) en el método login() de LoginViewModel.kt para validar el flujo de datos.

- **Evidencia Técnica:** Como se observa en la captura de pantalla adjunta a continuación, el depurador detuvo la ejecución de la aplicación en la línea 58.
- **Análisis de Variables:** Mediante el panel 'Threads & Variables', se navegó por el árbol de objetos hasta llegar al estado interno del MutableStateFlow. Se logró verificar en tiempo real que las variables user ("liliana") y pass ("123456") contenían los valores exactos ingresados en la interfaz de usuario.
- **Conclusión del Diagnóstico:** Esta prueba confirmó que la vinculación de datos (data binding) entre la vista (UI) y el modelo de vista (ViewModel) es íntegra y que no existen procesos intermedios que alteren la información antes de la autenticación.

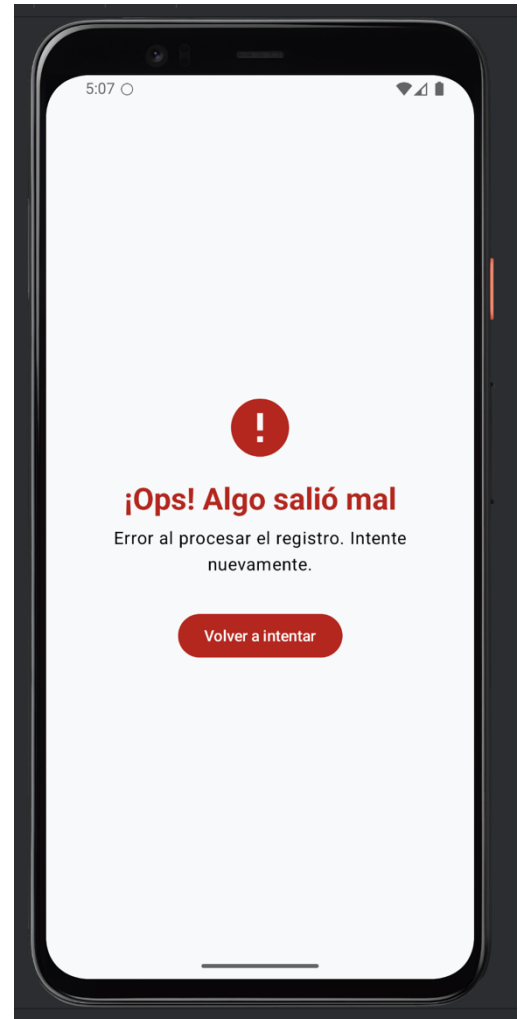


3. Manejo Estructurado de Excepciones

Se reforzó la robustez del sistema mediante la implementación de bloques try-catch estratégicos. Además, se extendió el sellado de estados ServiceState para incluir un estado de Error.

Acciones realizadas:

- Implementación de Try-Catch: Se envolvió todo el bloque de la corutina para capturar fallos inesperados en la persistencia o lógica.
- Feedback al Usuario: Se diseñó una pantalla de error en ResumenScreen.kt que muestra un mensaje útil y un botón de reintento, evitando que la app se cierre inesperadamente (crash).
- Prueba de Error: Se simuló una RuntimeException. El Logcat capturó el error en nivel Error (E) y la aplicación mostró la interfaz de contingencia correctamente.



```

2826-02-02 17:07:37.993 17987-17987 RegistroViewModel    cl.duoc.veterinaria
2826-02-02 17:07:37.996 17987-17987 RegistroViewModel    cl.duoc.veterinaria

D Iniciando procesarRegistro...
E Error durante el procesamiento del registro: ERROR CRÍTICO: El servidor de base de datos no responde. (Ask Gemini)
java.lang.RuntimeException: ERROR CRÍTICO: El servidor de base de datos no responde.
    at cl.duoc.veterinaria.ui.viewmodel.RegistroViewModel$procesarRegistro$1.invokeSuspend(RegistroViewModel.kt:123)
    at kotlin.coroutines.jvm.internal.BaseContinuationImpl.resumeWith(ContinuationImpl.kt:33) <9 internal lines>
    at cl.duoc.veterinaria.ui.viewmodel.RegistroViewModel.procesarRegistro(RegistroViewModel.kt:118)
    at cl.duoc.veterinaria.ui.registro.ResumenScreenKt$ResumenScreen$1$1.invokeSuspend(ResumenScreen.kt:41)
    at kotlin.coroutines.jvm.internal.BaseContinuationImpl.resumeWith(ContinuationImpl.kt:33) <1 internal line>
    at androidx.compose.ui.platform.AndroidUIDispatcher.performTrampolineDispatch(AndroidUIDispatcher.android.kt:79)
    at androidx.compose.ui.platform.AndroidUIDispatcher.access$performTrampolineDispatch(AndroidUIDispatcher.android.kt:41)
    at androidx.compose.ui.platform.AndroidUIDispatcher$dispatchCallback$1.run(AndroidUIDispatcher.android.kt:57)
    at android.os.Handler.handleCallback(Handler.java:95)
    at android.os.Handler.dispatchMessage(Handler.java:103)
    at android.os.Looper.loopOnce(Looper.java:240)
    at android.os.Looper.loop(Looper.java:338)
    at android.app.ActivityThread.main(ActivityThread.java:967) <1 internal line>
    at com.android.internal.os.RuntimeInit$MethodAndArgsCaller.run(RuntimeInit.java:593)
    at com.android.internal.os.ZygoteInit.main(ZygoteInit.java:932)

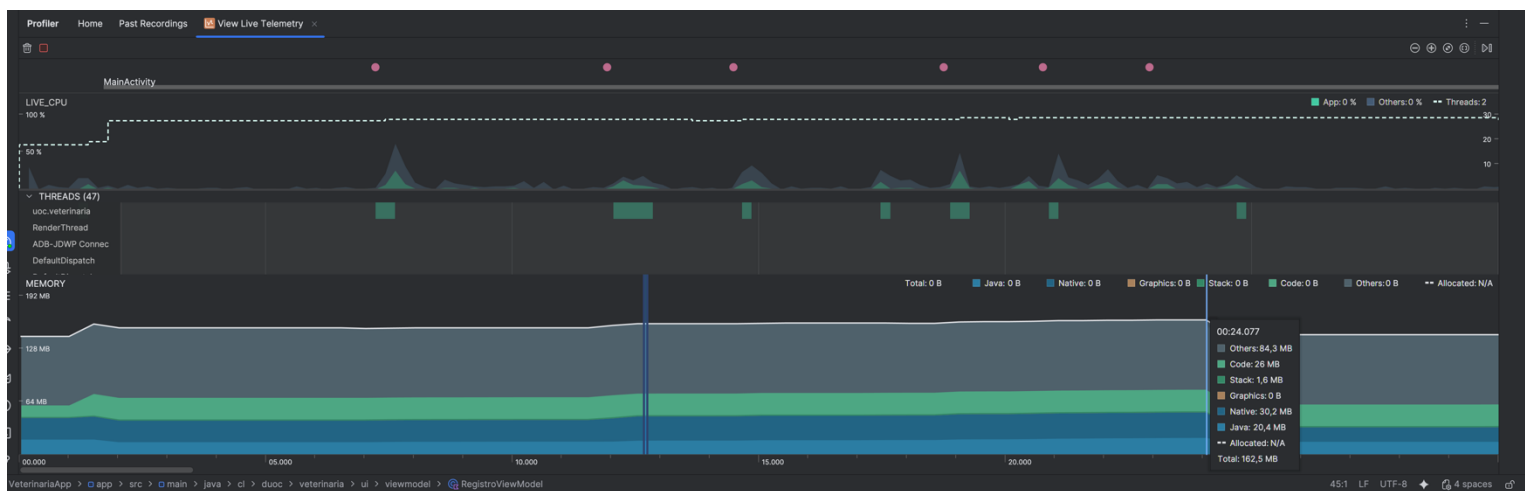
```

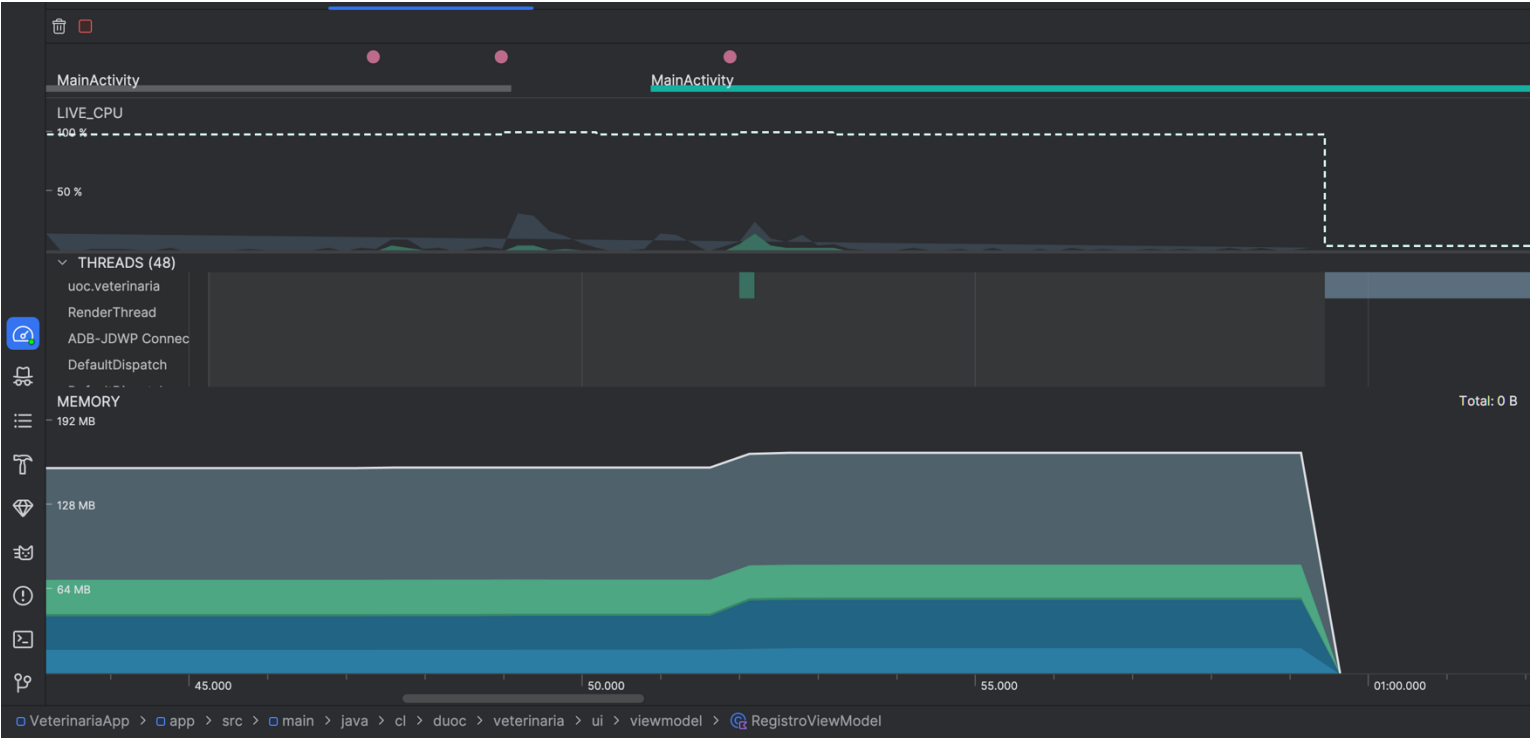
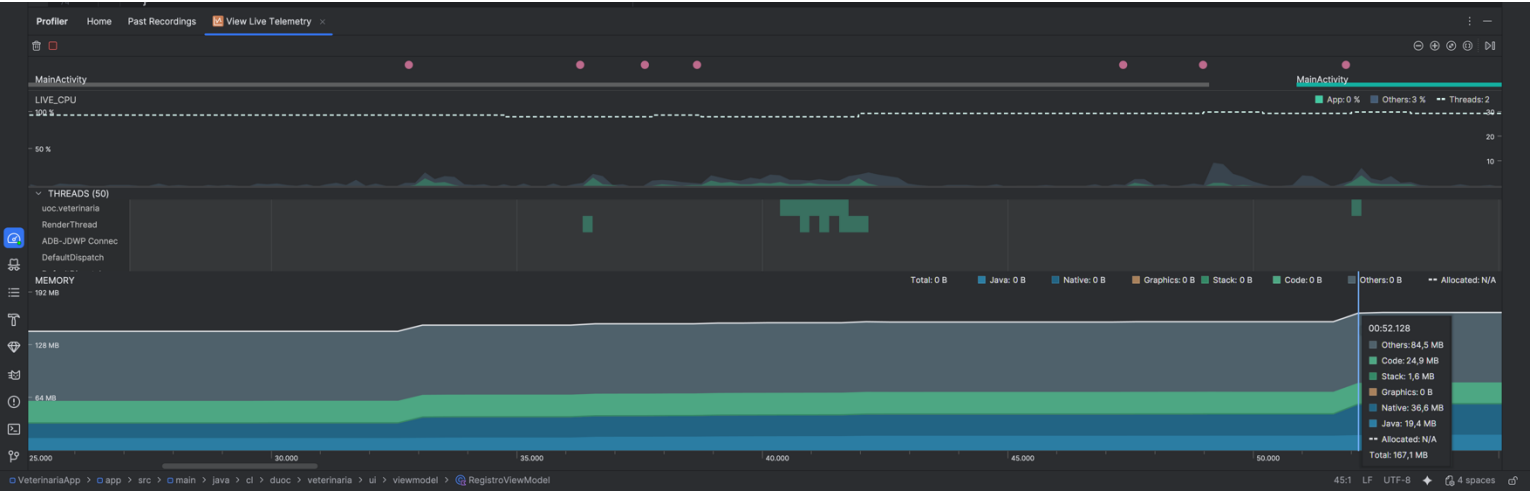
4. Análisis de Rendimiento en Tiempo de Ejecución

Se utilizó Android Profiler (Live Telemetry) para monitorear el consumo de recursos durante el flujo seleccionado.

Resultados del análisis:

- CPU: Se observaron picos de actividad menores durante la escritura en Room. La mayor parte del tiempo el uso de CPU fue mínimo, lo que confirma que las corutinas están suspendiendo el trabajo correctamente sin desperdiciar ciclos del procesador.
- Memoria (RAM): El consumo se mantuvo estable (entre 148MB y 167MB). No se detectaron fugas de memoria (memory leaks) ni crecimientos anómalos al navegar entre las pantallas de registro.
- Hilos (Threads): Se verificó que las tareas pesadas se ejecutan en hilos secundarios (DefaultDispatcher), manteniendo el hilo principal libre para la fluidez de la interfaz.





5. Reflexión Final

La implementación de estas herramientas y técnicas de manejo de errores ha tenido un impacto significativo en la aplicación:

- 1.Estabilidad: El manejo de excepciones garantiza que, ante un fallo de base de datos o red, el usuario reciba una explicación en lugar de un cierre forzado de la app.
- 2.Calidad Técnica: El uso de logs permite un mantenimiento mucho más rápido, ya que los desarrolladores pueden rastrear exactamente en qué paso falló el proceso.
- 3.Eficiencia: El análisis con Profiler confirmó que la aplicación es ligera y hace un uso responsable de la batería y memoria del dispositivo móvil, cumpliendo con los estándares de una app de alto rendimiento.

Cabe destacar que se identificaron diversos mensajes de sistema en el Logcat con etiquetas como `ashmem` o `Parcel`. Tras el análisis, se determinó que corresponden a advertencias de obsolescencia (deprecation) del framework de Android y no a errores en la lógica de negocio de la aplicación. El flujo crítico de la veterinaria no se ve afectado por estos mensajes, manteniendo su integridad y funcionalidad.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.